

Núcleo Cognitivo da Aurora Siger (NCAS)

Engenharia de Prompts e IA Generativa

Este documento detalha as estratégias de Engenharia de Prompt utilizadas no núcleo cognitivo, conforme desenvolvidas e simuladas no sistema.

1. Estratégia: Zero-Shot Prompting

Você é um núcleo de análise de alertas de uma missão espacial.

Você recebe alertas do modelo e precisa classificar a prioridade e a ação sugerida.

Classifique a prioridade da ocorrência abaixo:

"Falha crítica no sistema de oxigênio."

Prioridade: ALTA Ocorrência: Falha crítica no sistema de oxigênio. Ação sugerida: Acionar imediatamente os protocolos de emergência...

Explicação Contextual: O *Zero-Shot Prompting* consiste em instruir o Large Language Model (LLM) a executar uma tarefa sem fornecer nenhum exemplo de resposta anterior no corpo do prompt. O modelo baseia-se unicamente em seu pré-treinamento e no mecanismo de *Next Token Prediction*. Essa estratégia demonstra que o modelo da Aurora Siger já possui o conhecimento internalizado das prioridades de um suporte à vida, economizando tokens durante a inferência (barateando o custo por prompt).

2. Estratégia: Few-Shot Prompting

Você é um núcleo de análise de alertas de uma missão espacial.

Você recebe alertas do modelo e precisa classificar a prioridade e a ação sugerida.

Exemplo 1:

Ocorrência: "Luz de um corredor apagada."

Classificação: Baixa

Ação sugerida: Iniciar teste de religação...

Exemplo 2:

Ocorrência: "Falha no sistema de oxigênio."

Classificação: Alta

Ação sugerida: Ativar protocolos de emergência...

Agora classifique:

Ocorrência: "Oscilação no sistema de energia."

Classificação: Média Ação sugerida: Monitorar a oscilação e verificar os parâmetros de tensão...

Explicação Contextual: No *Few-Shot Prompting*, passamos deliberadamente um pequeno número de exemplos ("Exemplo 1" e "Exemplo 2") que formam um padrão. O LLM utiliza este contexto para se adaptar e entender exatamente o esquema esperado pela equipe para a saída. Isso auxilia a minimizar viés ou alucinações, garantindo que a resposta siga o mesmo tom e a mesma categorização (Baixa/Alta/Média) definida nas regras de negócio da missão espacial.

3. Estratégia: Structured Outputs (Saídas Estruturadas)

Você é um núcleo de análise de alertas de uma missão espacial.

Você recebe alertas do modelo e precisa classificar a prioridade e a ação sugerida.

Classifique a prioridade da ocorrência: "Falha crítica no sistema de oxigênio."

O retorno precisa ser no formato JSON abaixo:

```
{  
  "modulo": "",  
  "prioridade": "",  
  "acao_recomendada": "",
```

```
"risco": ""  
}
```

```
{ "modulo": "Energia", "prioridade": "Alta",  
  "acao_recomendada": "Acionar equipe de manutenção", "risco":  
  "Interrupção de módulos dependentes" }
```

Explicação Contextual: A técnica de *Structured Output* visa restringir o espaço de geração de texto do modelo, forçando a devolução estrita em formato de dicionário de dados (JSON). Isto é fundamental para a integração no fluxo de dados da colônia, pois permite que o código Python (utilizando o módulo `json` interno) acesse as informações via dicionário diretamente da memória, sem a necessidade de complexos parsers de texto ou falhas devido à verbosidade não solicitada da IA.